

# SAAS & WEB

## SaaS Fullstack Mastery: De 0 a Producción

Construye una plataforma SaaS completa desde cero: React/Next.js 14, Node.js, arquitectura multi-tenant, suscripciones con Stripe y MercadoPago.

[ Nivel: Intermedio ]

[ Duración: 24hs ]

[ 16 Módulos ]

[ Tech: Next.js 14, Node.js, MercadoPago, PostgreSQL ]

## #> Módulo 1: Arquitectura SaaS

---

El Software as a Service (SaaS) ha revolucionado la forma en que distribuimos aplicaciones. En lugar de vender licencias de un solo uso, cobramos suscripciones recurrentes.

### El Desafío Multi-Tenant

En un SaaS, cientos de empresas (inquilinos/tenants) usan tu misma aplicación simultáneamente. El mayor reto de arquitectura es garantizar que la Empresa A jamás pueda ver, por accidente, los datos de la Empresa B, a pesar de que ambos compartan la misma Base de Datos.

### Nuestro Stack Tecnológico

Para resolver esto construiremos una pila moderna:

- Frontend/Backend: Next.js 14 (App Router) permite hacer SEO, UI rápida y código de servidor en un solo proyecto.
- Base de Datos: PostgreSQL por su robustez en datos relacionales.
- ORM: Prisma o Drizzle para tipado seguro.
- Pagos: MercadoPago (Latam) y Stripe (Global).

## #> Módulo 2: Configuración del Entorno

---

Iniciemos nuestro proyecto desde cero. Usaremos las herramientas oficiales.

### Paso 1: Inicializar Next.js 14

1. Abre tu terminal.
2. Ejecuta el comando para crear el proyecto base. Nos hará varias preguntas.

```
npx create-next-app@latest mi-saas
# Selecciona:
# TypeScript: Yes
# ESLint: Yes
# Tailwind CSS: Yes
# `src/` directory: Yes
# App Router: Yes
```

3. Entra a la carpeta con ``cd mi-saas`` y ábrela en tu editor con ``code .``

### Paso 2: Base de Datos Local

En lugar de instalar PostgreSQL nativo que puede romper tu sistema, usaremos Docker para correrlo de forma limpia.

```
# Crea un archivo docker-compose.yml en la raíz:
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: password123
      POSTGRES_DB: saas_db
    ports:
      - "5432:5432"
```

Ejecuta ``docker-compose up -d`` en tu terminal para levantar la base de datos en segundo plano.

## #> Módulo 3: Modelado de Datos (Prisma ORM)

---

Un ORM nos permite escribir código JavaScript/TypeScript en lugar de SQL puro, autocompletando errores antes de que ocurran.

### Paso 1: Instalación de Prisma

1. Instala Prisma: ``npm install prisma --save-dev`` y ``npm install @prisma/client``.
2. Inicializa Prisma: ``npx prisma init``.

### Paso 2: El Esquema Multi-Tenant

Abre el archivo ``prisma/schema.prisma`` recién creado. Definiremos nuestro modelo fundamental: Organizaciones (Tenants) y Usuarios.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Tenant {
  id          String    @id @default(uuid())
  name        String
  stripeId    String?   // ID del cliente en Stripe
  plan        String    @default("FREE")
  users       User[]
  createdAt   DateTime  @default(now())
}

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  name        String?
  password    String
  tenantId    String
```



## #> Módulo 4: Autenticación con NextAuth

---

La seguridad no debe reinventarse. Usaremos NextAuth.js (Auth.js) para manejar sesiones encriptadas y login seguro.

### Paso 1: Instalación y Configuración

Instala ``npm install next-auth``.

```
// Crea el archivo: src/app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth"
import CredentialsProvider from "next-auth/providers/credentials"
import { prisma } from "@lib/prisma"

const handler = NextAuth({
  providers: [
    CredentialsProvider({
      name: "Credentials",
      credentials: {
        email: { label: "Email", type: "text" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        // 1. Buscar usuario en DB
        const user = await prisma.user.findUnique({ where: { email: credentials?.email } })

        if (!user) return null

        // 2. Verificar password (en prod, usa bcrypt.compare)
        if (user.password !== credentials?.password) return null

        // 3. Retornar datos, inyectando el Tenant ID
        return { id: user.id, name: user.name, email: user.email, tenantId: user.tenantId }
      }
    })
  ],
  callbacks: {
    session({ session, token }) {
      // Inyectar el tenantId en la sesión pública
      if (token && session.user) {
        session.user.tenantId = token.tenantId
      }
      return session
    }
  }
})
```



## #> Módulo 5: UI/UX con Tailwind y Shadcn UI

---

Tu SaaS debe verse premium. No escribiremos CSS a mano. Usaremos TailwindCSS (ya instalado) y Shadcn UI, una colección de componentes accesibles y hermosos.

### Paso 1: Inicializar Shadcn

1. En la terminal: ``npx shadcn-ui@latest init``
2. Te pedirá elegir un estilo (Default o New York) y colores. Elige tus favoritos.
3. Instala un botón: ``npx shadcn-ui@latest add button``.

### Paso 2: Tu primera vista

```
// src/app/page.tsx
import { Button } from "@/components/ui/button"

export default function LandingPage() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center bg-zinc-950 text-white p-24">
      <h1 className="text-5xl font-bold tracking-tight mb-6">
        El SaaS del Futuro
      </h1>
      <p className="text-zinc-400 mb-8 max-w-lg text-center">
        Optimiza tus procesos empresariales con nuestra plataforma impulsada por IA.
      </p>
      <Button size="lg" className="bg-emerald-500 hover:bg-emerald-600">
        Comenzar Prueba Gratis
      </Button>
    </main>
  )
}
```

Al guardar, Tailwind aplica las clases al instante y Shadcn provee un botón con animaciones y accesibilidad perfectas.

## #> Módulo 6: Protegiendo el Dashboard

---

El App Router de Next.js nos permite validar al usuario en el servidor antes de siquiera enviarle el HTML al navegador, mejorando la seguridad y velocidad.

### Creando una ruta protegida

Crea la carpeta `src/app/dashboard/page.tsx`.

```
import { getServerSession } from "next-auth"
import { redirect } from "next/navigation"
import { prisma } from "@/lib/prisma"

export default async function DashboardPage() {
  // 1. Obtener la sesión en el SERVIDOR
  const session = await getServerSession()

  // 2. Proteger la ruta: Si no hay sesión, echar al usuario
  if (!session || !session.user) {
    redirect("/api/auth/signin")
  }

  // 3. Consultar datos filtrando por el TENANT del usuario
  const tenantId = session.user.tenantId
  const tenantData = await prisma.tenant.findUnique({
    where: { id: tenantId }
  })

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Panel de Control</h1>
      <p>Bienvenido de vuelta, {session.user.name}</p>
      <div className="mt-4 p-4 border rounded">
        Empresa actual: {tenantData?.name}
        Plan: {tenantData?.plan}
      </div>
    </div>
  )
}
```

¡Observa cómo NUNCA enviamos el tenantId desde el cliente! Lo leemos de la cookie encriptada de sesión. Esto evita que un hacker altere el ID para ver datos de otra empresa.

## #> Módulo 7: Formularios y Server Actions

---

En React tradicional, necesitabas crear un endpoint en `/api/xxx`, usar `fetch()`, manejar estados de carga y errores. Next.js 14 simplifica esto con Server Actions.

### Paso 1: Crear una Server Action

```
// src/actions/actualizar-perfil.ts
"use server"

import { prisma } from "@lib/prisma"
import { getServerSession } from "next-auth"
import { revalidatePath } from "next/cache"

export async function actualizarNombre(formData: FormData) {
  const session = await getServerSession()
  if (!session) throw new Error("No autorizado")

  const nuevoNombre = formData.get("nombre") as str

  // Mutamos la base de datos directamente desde esta función
  await prisma.user.update({
    where: { email: session.user.email },
    data: { name: nuevoNombre }
  })

  // Refrescamos la UI sin recargar la página
  revalidatePath("/dashboard/perfil")
}
```

### Paso 2: Usar la Action en el Cliente

```
// src/app/dashboard/perfil/page.tsx
import { actualizarNombre } from "@actions/actualizar-perfil"
import { Button } from "@components/ui/button"

export default function Perfil() {
  return (
    <form action={actualizarNombre} className="flex flex-col gap-4 max-w-sm">
      <label>Tu Nombre</label>
```



## #> Módulo 8: Integración de Pagos (Stripe)

---

Para monetizar a nivel mundial, Stripe es el rey. Crearemos un flujo de suscripción mensual.

### Paso 1: Configurar Stripe

Instala ``npm install stripe``. Ve a tu dashboard de Stripe, crea un producto de suscripción y copia tu 'Secret Key' y el 'Price ID' del producto en tu ``.env``.

### Paso 2: Crear el Checkout de Suscripción

```
// src/app/api/checkout/route.ts
import Stripe from "stripe"
import { getServerSession } from "next-auth"
import { NextResponse } from "next/server"

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: Request) {
  const session = await getServerSession()
  if (!session) return new NextResponse("Unauthorized", { status: 401 })

  // Crear sesión de pago en Stripe
  const checkoutSession = await stripe.checkout.sessions.create({
    payment_method_types: ["card"],
    mode: "subscription", // Muy importante para pagos recurrentes
    customer_email: session.user.email,
    line_items: [
      {
        price: process.env.STRIPE_PRO_PRICE_ID,
        quantity: 1,
      },
    ],
    success_url: "http://localhost:3000/dashboard?exito=true",
    cancel_url: "http://localhost:3000/precios",
  })

  return NextResponse.json({ url: checkoutSession.url })
}
```



## #> Módulo 9: Pagos en Latam (MercadoPago)

---

Si tu público objetivo está en Latinoamérica, necesitarás integraciones locales como Pix, PagoFácil o cuotas. MercadoPago soluciona esto.

### Paso 1: Configurar el SDK

Instala `npm install mercadopago`. Obtén tu 'Access Token' en el panel de desarrolladores de MP.

```
// src/app/api/mp-checkout/route.ts
import { MercadoPagoConfig, Preference } from 'mercadopago';
import { NextResponse } from "next/server";

const client = new MercadoPagoConfig({ accessToken: process.env.MP_ACCESS_TOKEN! });

export async function POST() {
  const preference = new Preference(client);

  const response = await preference.create({
    body: {
      items: [
        {
          id: "plan_pro_anual",
          title: "Suscripción PRO Anual",
          quantity: 1,
          unit_price: 15000, // En moneda local (ej: ARS, MXN)
          currency_id: "ARS"
        }
      ],
      back_urls: {
        success: "http://localhost:3000/dashboard",
        failure: "http://localhost:3000/error"
      },
      auto_return: "approved"
    }
  });

  return NextResponse.json({ url: response.init_point });
}
```

## #> Módulo 10: Webhooks y Sincronización

---

El mayor error de novatos es asumir que si el Checkout tuvo éxito, el usuario ya pagó. Las tarjetas rebotan al renovarse. Necesitas Webhooks.

### Paso 1: Escuchar eventos de Stripe/MP

Un webhook es una URL en tu app que Stripe llama silenciosamente en el fondo cuando pasa algo importante (ej: ``invoice.paid`` o ``invoice.payment_failed``).

```
// src/app/api/webhooks/stripe/route.ts
import Stripe from 'stripe'
import { prisma } from '@lib/prisma'
import { NextResponse } from 'next/server'

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: Request) {
  const payload = await req.text()
  const sig = req.headers.get('stripe-signature') as string

  let event;
  try {
    // Validamos que el llamado realmente venga de Stripe
    event = stripe.webhooks.constructEvent(payload, sig,
process.env.STRIPE_WEBHOOK_SECRET!)
  } catch (err) {
    return new NextResponse("Webhook Error", { status: 400 })
  }

  // Sincronizar la base de datos
  if (event.type === 'invoice.payment_succeeded') {
    const invoice = event.data.object as Stripe.Invoice;
    // Buscamos a qué empresa (tenant) le pertenece este pago
    await prisma.tenant.update({
      where: { stripeId: invoice.customer as string },
      data: { plan: "PRO" }
    })
  }

  if (event.type === 'customer.subscription.deleted') {
    const sub = event.data.object as Stripe.Subscription;
    // Si canceló o la tarjeta falló mucho, lo bajamos al plan gratis
  }
}
```



## #> Módulo 11: Subdominios Multi-Tenant

---

Una gran experiencia SaaS es darle a cada cliente su propia URL (ej: apple.misaas.com, google.misaas.com).

### Configurando Next.js Middleware

Next.js permite interceptar requests antes de que lleguen a la página y reescribir la URL basándose en el subdominio.

```
// src/middleware.ts
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function middleware(req: NextRequest) {
  const hostname = req.headers.get('host') || ''

  // Extraer el subdominio (ej: apple de apple.misaas.com)
  const currentHost = hostname.replace('.misaas.com', '')

  // Si está accediendo a un subdominio válido
  if (currentHost !== 'www' && currentHost !== 'misaas.com') {
    // Reescribimos la URL internamente para que Next.js cargue
    // la carpeta /app/[tenant]/page.tsx
    return NextResponse.rewrite(new URL(`/${currentHost}${req.nextUrl.pathname}`,
req.url))
  }

  return NextResponse.next()
}

export const config = {
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*'],
}
```

## #> Módulo 12: Envío de Emails (Resend)

---

Tu SaaS necesita comunicarse. Enviaremos emails transaccionales cuando alguien se registre, pague o reinicie contraseña usando Resend y React Email.

### Enviando emails desde Server Actions

Instala ``npm install resend @react-email/components``. Resend permite escribir plantillas de email usando puro React.

```
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

export async function enviarBienvenida(emailUsuario: str, nombre: str) {
  await resend.emails.send({
    from: 'Soporte <soporte@misaas.com>',
    to: [emailUsuario],
    subject: '¡Bienvenido a la revolución SaaS!',
    // Usamos HTML o componentes React
    html: `

<h1>Hola ${nombre}</h1>
      <p>Gracias por registrarte en nuestra plataforma.</p>
      <a href="https://misaas.com/login">Inicia sesión aquí</a>
    </div>`
  });
}


```

## #> Módulo 13-16: Refinamiento y Despliegue

---

Los últimos módulos se centran en pulir y lanzar.

### Almacenamiento (S3 / Blob)

Para subir avatares, no uses tu base de datos. Usa Vercel Blob o AWS S3. Envías el archivo a S3 y guardas la URL pública (``https://s3.aws.../foto.jpg``) en PostgreSQL.

### Roles (RBAC)

Añade un campo ``role`` (ADMIN, MEMBER) al modelo User en Prisma. En tu servidor, verifica: ``if (session.user.role !== 'ADMIN') throw Error()`` antes de borrar facturas.

### Despliegue (Deploy)

Sube tu código a GitHub. Crea una cuenta en Vercel, enlaza tu repositorio. Vercel detectará que es Next.js y lo construirá automáticamente.

Para PostgreSQL, usa servicios serverless como Supabase, Neon o Render. Pega la URL de tu base de datos en las variables de entorno de Vercel.

¡Configura tu dominio final, actualiza los webhooks de Stripe con tu URL de producción y felicidades! Tu imperio SaaS ha comenzado.

# FULLSTACK SAAS ARCHITECT

Certificación otorgada a:

**[ DEVELOPER\_NAME ]**

Por haber dominado el diseño y desarrollo de plataformas Multi-Tenant, integrando bases de datos relacionales, autenticación robusta y pasarelas de pago recurrentes para llevar productos de 0 a Producción.

TIMESTAMP: 2026-08-12 // SYS\_ADMIN\_SIGNATURE